

1 Постановка задачи MSSP

Пусть дан ориентированный планарный граф

$$G = (V, E),$$

вложенный в плоскость, и фиксирована внешняя грань f_∞ .

Пусть:

- $c[d]$ — длина (стоимость) дуги d ;
- T_0 — дерево кратчайших путей, корень которого лежит на границе внешней грани.

Задача MSSP (Multiple-Source Shortest Paths):

Найти компактное представление всех деревьев кратчайших путей, корни которых расположены на границе внешней грани.

Главный результат:

MSSP решается за $O(n \log n)$.

2 Slack costs и price vectors

2.1 Определение slack cost

Пусть ρ — вектор потенциалов (price vector).

Определение 1. *Slack cost дуги d относительно ρ определяется как*

$$c_\rho[d] = c[d] + \rho(\text{tail}(d)) - \rho(\text{head}(d)).$$

Идея: потенциалы изменяют веса дуг, но сохраняют оптимальность путей.

Лемма 1. *Для любого пути P*

$$c_\rho(P) = c(P) + \rho(\text{start}(P)) - \rho(\text{end}(P)).$$

Следствие 1. *Кратчайшие пути сохраняются:*

для фиксированных s, t путь является кратчайшим относительно s тогда и только тогда, когда он кратчайший относительно c_ρ .

2.2 Relaxed, tense и tight дуги

Определение 2. *Дуга d называется:*

- *relaxed, если*

$$c_\rho[d] \geq 0;$$

- *tense, если*

$$c_\rho[d] < 0;$$

- *tight, если*

$$c_\rho[d] = 0.$$

2.3 Consistent price vectors

Определение 3. Вектор потенциалов ρ называется *consistent*, если все дуги *relaxed*:

$$\forall d \in E \quad c_\rho[d] \geq 0.$$

Лемма 2. Если $\rho(v) = \text{dist}(r, v)$, то:

1. ρ является *consistent*;
2. любая кратчайшая траектория из r состоит только из *tight*-дуг.

Следствие: если дерево состоит только из *tight*-дуг, то оно является деревом кратчайших путей.

Лемма 3. Если существует *consistent price vector*, то в графе нет отрицательных циклов.

3 Основная идея MSSP

Пусть

$$d_1, d_2, \dots, d_k$$

— дуги границы внешней грани f_∞ .

Для каждой вершины на границе строится дерево кратчайших путей:

$$T_0, T_1, T_2, \dots, T_k.$$

Алгоритм не хранит деревья полностью. Вместо этого хранится последовательность изменений между соседними деревьями.

3.1 Pivot

Определение 4. *Pivot* — операция:

- удалить одну дугу d^- из дерева;
- добавить другую дугу d^+ ;
- сохранить структуру дерева.

Есть два типа *pivot*:

1. **special pivot** — смена корня;
2. **ordinary pivot** — локальная перестройка дерева.

Главная идея MSSP:

$$T_i \rightarrow T_{i+1}$$

происходит через небольшое число *pivot*-операций.

4 Consecutive-Roots Lemma

Это ключевое свойство планарности.

Лемма 4 (Consecutive-Roots Lemma). *Для любой дуги d множество индексов*

$$\{i : d \in T_i\}$$

образует непрерывный отрезок на цикле границы.

Интуитивно: если дуга входит в дерево кратчайших путей для двух граничных корней, то она входит и для всех промежуточных.

Следствие 2. *Общее число ordinary pivots не превосходит числа дуг графа.*

Это обеспечивает линейное число изменений структуры.

5 Абстрактный алгоритм MSSP

Algorithm 1 MSSP

- 1: Вход: граф G , внешняя грань f_∞ , дерево T
 - 2: Пусть $d_1, \dots, d_s$ — дуги границы f_∞
 - 3: **for** $i = 1$ to s **do**
 - 4: Выполнить $\text{ChangeRoot}(T, d_i)$
 - 5: Получить множества удалённых и добавленных дуг
 - 6: **end for**
 - 7: Выход: последовательность pivot-операций
-

Сложность определяется количеством pivot-операций.

6 Процедура ChangeRoot

Основная подзадача MSSP — перейти от дерева с корнем в одной вершине к дереву с корнем в соседней вершине на границе.

Пусть:

$$\text{rev}(d_i)$$

— обратная дуга.

Алгоритм:

1. временно делает стоимость $\text{rev}(d_i)$ отрицательной;
2. выполняет special pivot;
3. постепенно увеличивает вес обратно;
4. при необходимости выполняет ordinary pivots.

6.1 Изменяемая стоимость

Определяется семейство стоимостей:

$$c_t[d] = \begin{cases} t, & d = \text{rev}(d_i), \\ c[d], & \text{иначе.} \end{cases}$$

Лемма 5. *Во время работы `ChangeRoot` дерево T всегда остаётся деревом кратчайших путей относительно c_t .*

7 Красно-синяя раскраска

Вершины делятся на:

- **blue** — путь к вершине проходит через $\text{rev}(d_i)$;
- **red** — иначе.

Лемма 6. *При увеличении параметра t :*

- *slack cost уменьшается для $\text{red} \rightarrow \text{blue}$ дуг;*
- *увеличивается для $\text{blue} \rightarrow \text{red}$ дуг;*
- *не меняется иначе.*

Это позволяет понять, какие дуги могут стать tense.

8 Эффективная реализация

Алгоритм хранит:

1. primal tree T ;
2. dual tree T^* .

Для dual tree используется структура данных:

Link-Cut Tree.

Поддерживаются операции:

- Cut;
- Link;
- Evert;
- AncestorFindMin;

- AddToAncestors.

Каждая pivot-операция требует:

$$O(\log n)$$

времени.

Так как число pivot-операций линейно, получаем итоговую сложность:

$$O(n \log n).$$

9 Вырожденный случай

Если кратчайшие пути не уникальны, возникают проблемы: одна и та же дуга может repeatedly входить и выходить из дерева.

Решение: выбирать **leafmost** tight-дугу.

Лемма 7. Пути к одной вершине после pivot-операций не пересекаются.

Лемма 8. Дуга не может снова войти в дерево во время одного вызова *ChangeRoot*.

Теорема 1. Каждая дуга удаляется из дерева не более одного раза за весь алгоритм.

Следовательно, общее число pivot-операций остаётся линейным.

10 Использование MSSP

10.1 Запросы кратчайших путей

После предобработки можно отвечать на запросы:

$$\text{ShortestPath}(u, v)$$

за

$$O(\ell \log \log n),$$

где ℓ — длина пути.

10.2 Запросы расстояний

Можно обрабатывать запросы:

$$\text{dist}(u, v)$$

для множества источников на границе.

Общая сложность:

$$O((n + q) \log n),$$

где q — число запросов.

10.3 Persistent data structures

Использование persistent link-cut tree позволяет хранить все версии деревьев.

Тогда запрос расстояния:

$$\text{dist}(u, v)$$

можно выполнять для любой версии дерева.

Размер структуры:

$$O(n \log n).$$

11 Основные идеи главы

1. Потенциалы (price vectors) переводят задачу к неотрицательным весам.
2. Tight-дуги полностью характеризуют кратчайшие пути.
3. Планарность даёт Consecutive-Roots property.
4. Изменения между соседними деревьями локальны.
5. MSSP использует pivot-операции аналогично network simplex.
6. Dual graph и link-cut trees позволяют реализовать алгоритм за

$$O(n \log n).$$

12 Итоговая асимптотика

Операция	Сложность
Один pivot	$O(\log n)$
Число pivot-ов	$O(n)$
MSSP полностью	$O(n \log n)$
ShortestPath query	$O(\ell \log \log n)$
Distance queries	$O((n + q) \log n)$